Notas de aula - MAT01050

Leonardo Suarez

13 de abril de 2026

Notas de aula da disciplina MAT01050 - Álgebra Matricial Computacional A, oferecida no semestre letivo 26/1. A bibliografia básica é composta por *Numerical Linear Algebra with Application Using MATLAB* de W. Ford, *Numerical Linear Algebra and Applications* de B. N. Datta, *Matrix Analysis and Applied Linear Algebra* de C. D. Meyer e *Algoritmos numéricos* de Campos Filho e Frederico Ferreira. O curso será ministrado pela profa. Liliane Basso Barichello e as aulas serão às terças e quintas-feiras, das 13:30 às 15:10.

O conteúdo programático é dividido em duas áreas. Primeiro, serão introduzidos os conceitos de normas matriciais, condicionamento de problemas e estabilidade de algoritmos. Em seguida, estudaremos métodos de solução de sistemas de equações lineares: eliminação gaussiana, fatoração LU e fatoração de Cholesky. Por fim, aboraremos transformações ortogonais, em especial a decomposição QR, as rotações de Givens e as reflexões de Householder. Na segunda área, usaremos as transformações ortogonais para resolver problemas de mínimos quadrados. Seguimos com o estudo de autovalores e autovetores. A disciplina se encerra com a introdução aos métodos iterativos: métodos estacionários de primeira ordem (Jacobi, Gauss-Seidel, SOR) e métodos de projeção (gradiente, gradientes-conjugados).

Teremos uma avaliação para cada área, com atividade presencial e entrega de tarefas ao decorrer do semestre letivo. A primeira avaliação será no dia 28 de abril/26 e a segunda no dia 30 de junho/26. Uma recuperação está prevista para o dia 07 de junho/26.

Conteúdo

I Aritmética de Ponto Flutuante **2**
1 Representação de inteiros 2
2 Representação de ponto flutuante 2
3 Aritmética de ponto flutuante 2
4 Minimizando erros 2

II Normas Vetoriais e Matriciais **2**
1 Normas vetoriais 2
2 Normas matriciais 3
3 Normas matriciais submultiplicativas 4

III Condicionamento de Problemas e Estabilidade de Algoritmos **4**
1 Erros computacionais 4
2 Condicionamento de um problema 4
3 Análise de perturbação para sistemas lineares 5
4 Propriedades do número de condicionamento de uma matriz 5

IV Sistemas de Equações Lineares **5**
1 Decomposição LU 5

2 Matrizes elementares 6
3 Eliminação gaussiana com pivotamento parcial 6
4 Custo computacional da decomposição LU 7
5 Estabilidade e eficiência da eliminação gaussiana 8
6 Fatoração de Cholesky 8

V Transformações Ortogonais **8**
1 Ortonormalização de Gram-Schmidt 8
2 Estabilidade numérica de Gram-Schmidt 8
3 A decomposição QR 8
4 Rotações de Givens 9
5 Reflexões de Householder 12

VI Problemas de mínimos quadrados **17**
1 Existência e unicidade de soluções de mínimos quadrados 17
2 Resolvendo problemas de mínimos quadrados sobredeterminados 17
3 Condicionamento de problemas de mínimos quadrados 17

VII Autovalores e Autovetores **17**
1 Teoremas de Gerschgorin 17
2 Método da potência para computar autovalor dominante 17
3 Iteração inversa de Hessenberg 17

VIII Métodos Iterativos **17**
1 Método de Jacobi 17
2 Método de Gauss-Seidel 17
3 Iteração SOR 17
4 Convergência dos métodos iterativos 17

I Aritmética de Ponto Flutuante

1 Representação de inteiros

2 Representação de ponto flutuante

3 Aritmética de ponto flutuante

4 Minimizando erros

II Normas Vetoriais e Matriciais

1 Normas vetoriais

Definição 1. Uma função $\|\cdot\| : \mathbb{R}^n \to \mathbb{R}$ é uma *norma vetorial* dado que:

1. $\|x\| \geq 0$ para todo $x \in \mathbb{R}^n$ e $\|x\| = 0$ se e somente se $x = 0$ (*positividade*);
2. $\|\alpha x\| = |\alpha|\|x\|$ para todo $\alpha \in \mathbb{R}$ (*homogeneidade*);
3. $\|x + y\| \leq \|x\| + \|y\|$ para todo $x, y \in \mathbb{R}^n$ (*desigualdade triangular*).

Define-se a *norma-p* como

$$\|x\|_p = \left(\sum_{i=1}^{n} |x_i|^p \right)^{1/p},$$

para $p \geq 1$. Os valores $p = 1, 2, \infty$[1] são as normas mais utilizadas, onde

$$\begin{aligned} \|x\|_1 &= \sum_{i=1}^{n} |x_i|, \\ \|x\|_2 &= \left(\sum_{i=1}^{n} |x_i|^2 \right)^{1/2}, \\ \|x\|_\infty &= \max_{1 \leq i \leq n} |x_i|. \end{aligned}$$

Lema (normas são equivalentes).

$$\begin{aligned} \|x\|_\infty \leq \|x\|_2 \leq \sqrt{n}\|x\|_\infty, \\ \|x\|_2 \leq \|x\|_1 \leq \sqrt{n}\|x\|_2, \\ \|x\|_\infty \leq \|x\|_1 \leq n\|x\|_\infty. \end{aligned}$$

A norma-2 é a mais usada em aplicações, pois goza diversas propriedades algébricas e geométricas como a *desigualdade de Cauchy-Schwarz*:

$$|\langle x, y \rangle| \leq \|x\|_2 \|y\|_2,$$

com a igualdade ocorrendo quando x e y são colineares. Outra relação que envolve a norma euclidiana é o *Teorema de Pitágoras* para x e y ortogonais:

$$\|x + y\|_2^2 = \|x\|_2^2 + \|y\|_2^2.$$

Por fim, a norma-2 é *ortogonalmente invariante*, isto é, para qualquer matriz P de ordem n ortogonal[2] vale

$$\|Px\|_2 = \|x\|_2,$$

para todo $x \in \mathbb{R}^n$. Sabendo que as colunas $p_1, \ldots p_n$ de P ortogonal formam base ortonormal[3] de $\mathbb{R}^n$, a propriedade acima indica que o comprimento de x não se altera na mudança de base da base canônica (que é ortonormal) para a base ortonormal $p_1, \ldots p_n$.

2 Normas matriciais

Usamos normas vetoriais para mensurar o comprimento de um vetor. Similarmente, definimos normas matriciais para medir o tamanho de matrizes. O tamanho de uma matriz é usado para avaliar se a solução x de um sistema linear $Ax = b$ pode ser confiável, para analisar a taxa de convergência de uma sequência de vetores etc. A definição de norma matricial é análoga à definição de norma vetorial, mas agora a função $\| \cdot \|$ é definida sobre o espaço de matrizes $\mathbb{R}^{m \times n}$.

[1] As normas $p = 2$ e $p = \infty$ são chamadas de *norma euclidiana* e *norma do máximo*, respectivamente.

[2] Uma matriz ortogonal (ou ortonormal) Q é uma matriz real simétrica cujas linhas e colunas são vetores ortonormais, ou seja, $Q^T Q = QQ^T = I$.

[3] Prova-se que P é ortogonal sse suas colunas são ortonormais. Daí se $p_1, \ldots, p_k \in \mathbb{R}^n$ são ortogonais então formam base dum subespaço de dimensão k de $\mathbb{R}^n$

Definição 2. Uma função $\|\cdot\| : \mathbb{R}^{m\times n} \to \mathbb{R}$ é uma *norma matricial* se satisfaz as seguintes propriedades:

1. $\|A\| \geq 0$ para toda $A \in \mathbb{R}^{m\times n}$ e $\|A\| = 0$ se e somente se $A = 0$ (*positividade*);
2. $\|\alpha A\| = |\alpha|\|A\|$ para todo $\alpha \in \mathbb{R}$ (*homogeneidade*);
3. $\|A + B\| \leq \|A\| + \|B\|$ para todo $A, B \in \mathbb{R}^{m\times n}$ (*desigualdade triangular*).

Uma das normas matriciais mais simples é a *norma de Frobenius*, ou também *norma de Hilbert-Schmidt*. Ela é definida como a raiz quadrada da soma dos quadrados de todas as entradas da matriz, isto é,

$$\|A\|_F = \sqrt{\sum_{i=1}^{m}\sum_{j=1}^{n} |a_{ij}|^2}.$$

A classe de normas matriciais mais usadas são as *normas matriciais induzidas*, que são definidas em termos de uma norma vetorial.

Definição 3. Seja $\|\cdot\|$ uma norma vetorial, A uma matriz $m \times n$ e x um vetor $n \times 1$. Então a *norma matricial induzida* por $\|\cdot\|$ é dada por

$$\|A\| = \max_{x\neq 0} \frac{\|Ax\|}{\|x\|} = \max_{\|x\|=1} \|Ax\|. \tag{1}$$

3 Normas matriciais submultiplicativas

III Condicionamento de Problemas e Estabilidade de Algoritmos

1 Erros computacionais

2 Condicionamento de um problema

Definição 1. Um problema é *mal-condicionado* se uma pequena perturbação nos dados de entrada pode[4] causar uma grande perturbação na solução, independentemente do algoritmo usado para resolver o problema. Se pequenas perturbações nos dados de entrada causam pequenas perturbações na solução, dizemos que o problema é *bem-condicionado*.

Definição 2. Sejam x e $\overline{x}$ as soluções exata e aproximada, e $f(x)$ e $f(\overline{x})$ as respectivas soluções. Então,

- O problema é *bem-condicionado* com respeito a x se sempre que $|x - \overline{x}|$ é pequeno, $|f(x) - f(\overline{x})|$ é pequeno;
- O problema é *mal-condicionado* com respeito a x se existe uma perturbação arbitrariamente pequena $|x - \overline{x}|$ tal que $|f(x) - f(\overline{x})|$ é grande.

Definição 3. O *número de condicionamento* de um problema f com entrada x é

$$C_f(x) \stackrel{\text{def}}{=} \lim \varepsilon \to^+ \sup_{\delta x \leq \varepsilon} \frac{\frac{\|f(x+\delta x) - f(x)\|}{\|f(x)\|}}{\frac{\|\delta x\|}{\|x\|}} \tag{2}$$

[4]Um problema mal-condicionado ainda assim pode ter solução satisfatória.

Observação. O *supremo* (sup) de um subconjunto S de um conjunto ordenado X é o menor elemento de X que é maior ou igual a todos os elementos de S. O supremo difere do máximo (max) de um conjunto S porque o supremo pode não pertencer a S.

Se f é diferenciável, podemos descrever $C_f(x)$ em termos de f' usando o teorema do valor médio.

Teorema 1 (teorema do valor médio). *Se uma função $f : [a, b] \to \mathbb{R}$ é contínua em $[a, b]$ e diferenciável em (a, b), então existe um ponto $\xi \in (a, b)$ tal que*

$$f'(\xi) = \frac{f(b) - f(a)}{b - a}.$$

Fixa x e faz $\overline{x} = x + \delta x$. Aplica (1) e obtém $C_f(x) = |x||f'(x)|/|f(x)|$.

3 Análise de perturbação para sistemas lineares

Teorema 2. *Sejam A uma matriz não singular, $b \neq 0$ um vetor, x a solução para $Ax = b$ e $\|\cdot\|$ uma norma subordinada.*

1. *Se $x + \delta x$ é solução para o sistema perturbado $A(x + \delta x) = b + \delta b$, então*

$$\frac{\|\delta x\|}{\|x\|} \leq \|A\|\|A^{-1}\|\frac{\|\delta b\|}{\|b\|}. \tag{3}$$

2. *Se $x + \delta x$ é solução para o sistema perturbado $(A + \delta A)(x + \delta x) = b$, então*

$$\frac{\|\delta x\|}{\|x + \delta x\|} \leq \|A\|\|A^{-1}\|\frac{\|\delta A\|}{\|A\|}. \tag{4}$$

3. *Se $x + \delta x$ é solução para o sistema perturbado $(A + \delta A)(x + \delta x) = b + \delta b$, então*

$$\frac{\|\delta x\|}{\|x\|} \leq \|A\|\|A^{-1}\|\left(\frac{\|\delta A\|}{\|A\|} + \frac{\|\delta b\|}{\|A\|\|x + \delta x\|}\right) \tag{5}$$

Definição 4. O número $\|A\|\|A^{-1}\|$ é chamado de *número de condicionamento* de A e é denotado por $\kappa(A)$.

Observação. Qualquer norma matricial pode ser usada para computar o número de condicionamento. As notações $\kappa_\infty(A)$, $\kappa_1(A)$ e $\kappa_F(A)$ se referem às normas $\infty-$, $1-$ e $F-$, respectivamente.

Definição 5. Se o número de condicionamento da matriz A é grande[5], dizemos que A é *mal-condicionada*; caso contrário, A é *bem-condicionada.*

4 Propriedades do número de condicionamento de uma matriz

IV Sistemas de Equações Lineares

1 Decomposição LU

Por ora, sem considerar trocas de linhas, podemos fatorar uma matriz A de ordem n no produto de uma matriz triangular-inferior L e uma matriz triangular-superior U. Faz $L = I$. Aplica a eliminação

[5]Imediatamente se deve perguntar o que é um número de condicionamento grande. A resposta depende do problema analisado.

gaussiana em A para obter a matriz triangular-superior U. Sendo a_{ii} o elemento pivô, coloca os multiplicadores $a_{i+1,i}/a_{ii}, a_{i+2}/a_{ii}, \ldots, a_{ni}/a_{ii}$, com $1 \leq i \leq n-1$, nas posições $(i+1, i), (i+2, i), \ldots, (n, i)$ da matriz L. Ao final da eliminação, tem-se $A = LU$.

Dada a decomposição $A = LU$, podemos resolver o sistema $Ax = b$. Substitui A por LU e fica

$$LUx = b \tag{6}$$
$$L(Ux) = b. \tag{7}$$

Faz $y = Ux$ e resolve $Ly = b$ usando substituição para frente (ou *forward substitution*). Depois resolve $Ux = y$ com substituição para trás (ou*back substitution*).

2 Matrizes elementares

Definição 1. Para cada uma das três operações elementares de linha, existe uma *matriz elementar* E_{ij}, $E_i(t)$ e $E_{ij}(t)$:

1. E_{ij} é a matriz que troca as linhas i e j da matriz identidade I;
2. $E_i(t)$ é a matriz que multiplica a linha i da matriz identidade I por um escalar $t \neq 0$;
3. $E_{ij}(t)$ é a matriz que subtrai t vezes a linha j da linha i da matriz identidade I.

Proposição 1. *Se uma matriz A $n \times n$ é multiplicada à esquerda por uma matriz elementar E, então o resultado é a matriz obtida pela aplicação da operação elementar correspondente à E nas linhas da matriz A.*

Proposição 2. *Matrizes elementares são não singulares; de fato*

1. $E_{ij}^{-1} = E_{ij}$;
2. $E_i(t)^{-1} = E_i(1/t)$;
3. $E_{ij}(t)^{-1} = E_{ij}(-t)$.

Proposição 3. *Se B é linha-equivalente a A, então A é linha-equivalente a B.*

Proposição 4. *Seja A uma matriz não singular $n \times n$. Então*

1. *A é linha-equivalente à matriz identidade I;*
2. *A é produto de matrizes elementares.*

Proposição 5. *Seja A uma matriz $n \times n$ e suponha que A é linha-equivalente a I. Então A é não singular e A^{-1} pode ser obtida por meio da aplicação das mesmas operações elementares usadas para transformar A em I.*

Proposição 6. *A decomposição LU de uma matriz A não singular é única.*

3 Eliminação gaussiana com pivotamento parcial

O algoritmo da eliminação gaussiana pode gerar problemas se os elementos pivôs são pequenos. Os erros ocorrem quando dividimos por elementos pivôs pequenos e obtemos multiplicadores a_{ji}/a_{ii} grandes. Esses multiplicadores, quando somados a elementos pequenos, resultam numa perda de contribuição dos números menores. A solução para esse problema é escolher o elemento com maior magnitude em

cada coluna como pivô, de modo que $|a_{ji}/a_{ii}| \leq 1$ para $i+1 \leq j \leq n$. Chamamos essa estratégia de *eliminação gaussiana com pivotamento parcial* (EGPP).

Se usarmos EGPP, a decomposição LU de A consiste em três matrizes P, L e U tal que

$$PA = LU, \tag{8}$$

onde P é uma *matriz de permutação*, ou *matriz pivô*, que representa as trocas de linha feitas durante o processo de eliminação.

O seguinte código realiza a decomposição LU com pivotamento parcial:

Listing 1: Decomposição LU com pivotamento parcial

```
function [L,U,P] = ludecomp(A)
    // performs LU decomposition using gaussian elimination with partial
        pivoting
    // input: n x n matrix A
    // output: lower-triangular matrix L, upper-triangular matrix U, and
        permutation matrix P such that PA=LU
    n = size(A,1);
    U = A;
    L = eye(n,n);
    P = eye(n,n);
    for i = 1:n-1
        [m,k] = max(abs(U(i:n,i))); // get the largest pivot in each
            column (value m, index k)
        pivot_index = k+i-1;
        //perform row exchanges if current row is not a pivot row
        if pivot_index ~= i then
            U([i,pivot_index],:) = U([pivot_index,i],:); // exchange rows
                i and pivot_index in matrix A
            P([i,pivot_index],:) = P([pivot_index,i],:);
            if i > 1 then
                L([i,pivot_index],1:i-1) = L([pivot_index,i],1:i-1); //
                    exchange rows before the current column in matrix L
            end
        end
        multipliers = U(i+1:n,i) / U(i,i);
        U(i+1:n,i+1:n) = U(i+1:n,i+1:n) - multipliers*U(i,i+1:n); //
            update submatrix
        L([i+1:n],i) = multipliers;
    end
    U = triu(U);
endfunction
```

4 Custo computacional da decomposição LU

Se o sistema linear $Ax_i = b_i$, $1 \leq i \leq k$, precisa ser resolvido para múltiplos vetores b_i, podemos realizar a decomposição $PA = LU$, de custo computacional $O(n^3)$, apenas uma vez. Depois resolvemos $L(Ux_i) = Pb_i$ usando substituição para frente e substituição para trás, que têm custo $O(kn^2)$. Caso fôssemos realizar a eliminação gaussiana para cada sistema $Ax_i = b_i$, o custo seria $O(kn^3)$.

Dada uma decomposição LU de uma matriz A, podemos resolver o sistema $Ax = b$ da seguinte forma:

Listing 2: Resolução de sistema linear usando decomposição LU

```
function [x] = lusolve(L,U,P,b)
    // solve linear system given LU decomposition
    // input: lower-triangular matrix L, upper-triangular matrix U,
        permutation matrix P, and vector b
    // output: solution vector x
    n = size(b,1);

    b = P*b; // perform same row exchanges from LU factorization in the
        vector b
    // solve Ly = b
    y = zeros(n,1); //initialize vector y
    y(1) = b(1);
    for i = 2:n
        y(i) = (b(i) - L(i,1:i-1)*y(1:i-1)) / L(i,i); // perform forward
            substitution
    end
    // solve Ux = y
    x = zeros(n,1); // initialize solution vector x
    x(n) = y(n) / U(n,n);
    for k = n-1:-1:1
        x(k) = (y(k) - U(k,k+1:n)*x(k+1:n)) / U(k,k); // perform back
            substitution
    end
endfunction
```

5 Estabilidade e eficiência da eliminação gaussiana

Definição 2. Aplica a fatoração LU na matriz A. Durante os passos da eliminação, temos as matrzes $A = A^{(0)}, A^{(1)}, \ldots, A^{(n-1)} = U$. O *fator de crescimento* é a razão

$$\rho \stackrel{\text{def}}{=} \frac{\max_{i,j,k} |a_{ij}^{(k)}|}{\max_{i,j} |a_{ij}|},$$

em outras palavras, é a razão entre o maior elemento que aparece durante a eliminação gaussiana e o maior elemento da matriz A.

6 Fatoração de Cholesky

V Transformações Ortogonais

1 Ortonormalização de Gram-Schmidt

2 Estabilidade numérica de Gram-Schmidt

3 A decomposição QR

Teorema 1. *Se A é uma matriz $m \times n$, $m \geq n$, com colunas linearmente independentes, então existe uma matriz Q $m \times n$ com colunas ortonormais e uma matriz R $n \times n$ triangular superior tal que $A = QR$.*

4 Rotações de Givens

Na decomposição LU, multiplicamos uma matriz A à esquerda por uma sequência de matrizes elementares para obter $A = LU$, onde L é um produto de matrizes elementares e U é triangular-superior. Similarmente, a ideia aqui é realizar a fatoração $A = QR$ através da multiplicação à esquerda de A por uma sequência de *matrizes de Givens* e obter uma matriz ortogonal Q e uma matriz triangular-superior R. Cada multiplicação por uma matriz de Givens zera uma entrada da matriz original para construir R. O seguinte esquema ilustra o algoritmo a ser estudado:

Algoritmo da decomposição QR de Givens

Aplica $n-1$ matrizes de Givens, J_{i1}, $2 \leq i \leq n$, à esquerda de A para zerar as entradas $a_{21}, a_{31}, \ldots, a_{n1}$

$$(J_{n1}J_{n-1,1}\ldots J_{21})A \to \begin{bmatrix} X & X & \ldots & X \\ 0 & X & \ldots & X \\ 0 & X & \ldots & X \\ \vdots & \vdots & \vdots & \vdots \\ 0 & X & \ldots & X \end{bmatrix}$$

Agora usa $n-2$ matrizes de Givens para zerar os elementos nos índices $(3,2), (4,2), \ldots, (n,2)$ para obter

$$(J_{n2}J_{n-1,2}\ldots J_{42}J_{32})(J_{n1}J_{n-1,1}\ldots J_{21})A \to \begin{bmatrix} X & X & \ldots & X \\ 0 & X & \ldots & X \\ 0 & 0 & \ldots & X \\ \vdots & \vdots & \vdots & \vdots \\ 0 & 0 & \ldots & X \end{bmatrix}$$

Seja J_i o produto de matrizes Givens usado para zerar os elementos abaixo da entrada a_{ii}. Continua o processo até obter R triangular superior

$$(J_{n-1}J_{n-2}\ldots J_2J_1)A \to \begin{bmatrix} X & X & \ldots & X & X \\ 0 & X & \ldots & X & X \\ 0 & 0 & X & \ldots & X \\ \vdots & \vdots & \vdots & \ddots & \vdots \\ 0 & 0 & 0 & \ldots & X \end{bmatrix} = R,$$

e

$$A = (J_{n-1}J_{n-2}\ldots J_2J_1)^T R = QR.$$

onde Q é o produto de matrizes ortogonais, logo ortogonal[a].

[a] Se $P = P_kP_{k-1}\ldots P_2P_1$, onde P_i é ortogonal para $1 \leq i \leq k$, então P é ortogonal, pois $P^TP = P_1^TP_2^T\ldots P_k^TP_kP_{k-1}\ldots P_2P_1 = I$.

Definição 1. Uma matriz da forma

$$J(i,j,c,s) = \begin{bmatrix} 1 & 0 & 0\dots & \dots & \dots & \dots & 0 & \\ 0 & 1 & 0 & \dots & \dots & \dots & \dots & 0 \\ \vdots & \vdots & \vdots & & & & & \\ 0 & 0 & 0 & \dots & c & s & \dots & 0 \\ \vdots & \vdots & \vdots & & & & & \\ 0 & 0 & 0 & \dots & -s & c & \dots & 0 \\ \vdots & \vdots & \vdots & & & & & \\ 0 & 0 & 0 & \dots & 0 & \dots & 1 & \end{bmatrix},$$

é uma *matriz de Givens*. O valor c está na diagonal nos índices (i,i) e (j,j), $i < j$. O valor s está na posição (i,j) e o valor $-s$ na posição (j,i). Os elementos restantes da diagonal são 1 e os elementos fora da diagonal são 0.

Queremos que uma matriz de Givens seja ortogonal ($J(i,j,c,s)^T J(i,j,c,s) = I$), e claramente suas colunas são ortogonais. Basta então que tenham comprimento unitário, ou seja, que se tenha $c^2+s^2 = 1$. Para visualizar isso, considera a matriz de Givens

$$J(1,3,c,s) = \begin{bmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{bmatrix}.$$

Computa o produto $J(1,3,c,s)^T J(1,3,c,s)$ e verifica que $c^2+s^2 = 1$ é necessário para que $J(1,3,c,s)$ seja ortogonal:

$$J(1,3,c,s)^T J(1,3,c,s) = \begin{bmatrix} c & 0 & -s \\ 0 & 1 & 0 \\ s & 0 & c \end{bmatrix} \begin{bmatrix} c & 0 & s \\ 0 & 1 & 0 \\ -s & 0 & c \end{bmatrix} = \begin{bmatrix} c^2+s^2 & 0 & 0 \\ 0 & 1 & 0 \\ 0 & 0 & c^2+s^2 \end{bmatrix}.$$

Usando a identidade trigonométrica $\cos^2\theta + \sin^2\theta = 1$, podemos tomar $c = \cos\theta$ e $s = \sin\theta$ e obter uma *matriz de rotação de Givens*, denotada por $J(i,j,\theta)$. No $\mathbb{R}^2$, a matriz de Givens $J(1,2,\theta)$ é $\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$.

O algoritmo da decomposição QR de Givens depende de zerar elementos em posições específicas de uma matriz. Seja $x = [x_1 x_2 \cdots x_{n-1} x_n]^T$ e calcula o produto $J(i,j,c,s)x$.

$$J(i,j,c,s)x = \begin{bmatrix} 1 & 0 & 0\dots & \dots & \dots & \dots & 0 & \\ 0 & 1 & 0 & \dots & \dots & \dots & \dots & 0 \\ \vdots & \vdots & \vdots & & & & & \\ 0 & 0 & 0 & \dots & c & s & \dots & 0 \\ \vdots & \vdots & \vdots & & & & & \\ 0 & 0 & 0 & \dots & -s & c & \dots & 0 \\ \vdots & \vdots & \vdots & & & & & \\ 0 & 0 & 0 & \dots & 0 & \dots & 1 & \end{bmatrix} \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ x_i \\ \vdots \\ x_j \\ \vdots \\ x_n \end{bmatrix} = \begin{bmatrix} x_1 \\ x_2 \\ \vdots \\ cx_i + sx_j \\ \vdots \\ -sx_i + cx_j \\ \vdots \\ x_n \end{bmatrix}. \tag{9}$$

Nota que o produto $J(i,j,c,s)x$ altera somente as componentes i e j do vetor x. Então, se quisermos zerar a componente x_j, $i < j$, podemos reduzir o problema de encontrar c e s para o caso bidimensional.

Toma a matriz de Givens $\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix}$ que rotaciona o vetor $\begin{bmatrix} x \\ y \end{bmatrix} \in \mathbb{R}^2$ sobre o eixo x, zerando a coordenada y. Do plano cartesiano, é fácil ver que $\cos\theta = x/\sqrt{x^2+y^2}$ e $\sin\theta = y/\sqrt{x^2+y^2}$, então

$$\begin{bmatrix} \cos\theta & \sin\theta \\ -\sin\theta & \cos\theta \end{bmatrix} \begin{bmatrix} x \\ y \end{bmatrix} = \begin{bmatrix} \frac{x}{\sqrt{x^2+y^2}} & \frac{y}{\sqrt{x^2+y^2}} \\ -\frac{y}{\sqrt{x^2+y^2}} & \frac{x}{\sqrt{x^2+y^2}} \end{bmatrix} = \begin{bmatrix} \sqrt{x^2+y^2} \\ 0 \end{bmatrix}.$$

Generalizando o argumento, para zerar x_j, escolha $i < j$ e calcula os valores

$$c = \frac{x_i}{\sqrt{x_i^2 + x_j^2}}, \quad s = \frac{x_j}{\sqrt{x_i^2 + x_j^2}}.$$

Faz $J(i, j, c, s) = I$ seguido de

$$\begin{aligned} J(i, i, c, s) &= J(j, j, c, s) = c \\ J(i, j, c, s) &= s \\ J(j, i, c, s) &= -s \end{aligned}$$

e computa $J(i, j, c, s)x$.

Segue-se que dado um vetor $x = [x_1 x_2 \dots x_n]^T$, podemos usar um produto de matrizes Givens para zerar os $n - i$ elementos de x abaixo da entrada x_i. Para zerar x_{i+1}, computa $J(i, i+1, c_{i+1}, s_{i+1})x = \overline{x_{i+1}}$. Para zerar x_{i+2}, computa $J(i, i+2, c_{i+2}, s_{i+2})\overline{x_{i+1}} = \overline{x}_{i+2}$ e continua o processo até $J(i, n, c_n, s_n)\overline{x_{n-1}} = \overline{x_n}$. Em suma, o produto

$$J(i, n, c_n, s_n) \dots J(i, i+2, c_{i+2}, s_{i+2}) J(i, i+1, c_{i+1}, s_{i+1}) x$$

transforma x num vetor da forma $[x_1 x_2 \dots x_{i-1} \star 0 \dots 0]^T$.

Observação. As operações com matrizes de Givens podem ser feitas implicitamente, isto é, não é necessário construí-las.

Se $J(i, j, c, s)$ é uma matriz $m \times m$ e A é uma matriz $m \times n$, o produto $J(i, j, c, s)A$ altera somente as linhas i e j. De (9), vemos que as linhas i e j, $i < j$, têm forma

i: $ca_{i1} + sa_{j1}, ca_{i2} + sa_{j2}, \dots, ca_{in} + sa_{jn}$

j: $ca_{j1} - sa_{i1}, ca_{j2} - sa_{i2}, \dots, ca_{jn} - sa_{in}$

Isso faz com que o produto $J(i, j, c, s)A$ seja computado em apenas $6n$ operações. O algoritmo abaixo faz a multiplicação de A por uma matriz de Givens, dados os parâmetros c e s.

Listing 3: Multiplicação de uma matriz por uma matriz de Givens

```
function A = givensmul(A,i,j,c,s)
// multiplication of m x n matrix A by Givens matrix
// input: matrix A, indices i and j, Givens parameters c and s
// output: m x n matrix J(i,j,c,s)A

a = A(i,:);
b = A(j,:);

A(i,:) = c*a + s*b;
A(j,:) = -s*a + c*b;
endfunction
```

O cálculo dos parâmetros c e s pode ter problemas de *overflow* ou *underflow*. Uma maneira de minimizar esses erros é normalizar as entradas x_i e x_j, conforme o seguinte algoritmo.

Listing 4: Cálculo dos parâmetros de Givens

```
function [c,s] = givensparms(x_i,x_j)
    // normalize the entries x_i or x_j to minimize overflow/underflow
        errors
    // input: x_i and x_j, j>i, of vector x
    // output: Givens parameters c and s for x

    if x_j == 0 then
        c=1;
        s=0;
    elseif abs(x_j) > abs(x_i) then
        t=x_i/x_j; // make t such that 0<t<1
        s=1/sqrt(1+t^2);
        c=s*t;
    else
        t=x_j/x_i;
        c=1/sqrt(1+t^2);
        s=c*t;
    end
endfunction
```

Definição 2. Uma matriz $A = [a_{ij}]$ $m \times n$ é *triangular-superior* se $a_{ij} = 0$ para $i > j$. Em outras palavras, A é triangular-superior se os elementos abaixo da diagonal principal são nulos.

O algoritmo abaixo realiza a decomposição QR de Givens.

Listing 5: Decomposição QR de Givens

```
function [Q,R] = givensqr(A)
    // compute the QR decomposition of A
    // input: m x n matrix A
    // output: m x n orthogonal matrix Q
    // and m x n upper-triangular matrix R
    [m,n] = size(A);
    Q = eye(m,m); // initialize Q = I

    for i = 1:min(m-1,n)
        for j = i+1:m // run through column i
            [c,s] = givensparms(A(i,i),A(j,i));
            A = givensmul(A,i,j,c,s);
            Q = givensmul(Q,i,j,c,s);
        end
    end
    R = A;
    Q = Q';
endfunction
```

5 Reflexões de Householder

As reflexões de Householder são uma alternatiiva às rotações de Givens para computar a decomposição QR de uma matriz. Uma rotação de Givens zera um elemento de cada vez, e é preciso de uma sequência

de rotações para transformar uma matriz em forma triangular-superior. A reflexão de Householder, por outro lado, consegue zerar todos os elementos abaixo de um elemento da diagonal a_{ii} com uma única multiplicação, e por isso seu algoritmo para a decomposição QR requer cerca de dois-terços do número de *flops* do algoritmo de Givens. Entretanto, é importante notar que rotações de Givens são paralelizáveis (ao contrário das reflexões de Householder); além disso, em alguns casos, pode ser mais eficiente eliminar uma entrada por vez, situação que é mais adequada ao algoritmo de Givens.

Definição 3. Uma *reflexão de Householder* (ou *transformação de Householder*) H_u é uma transformação que reflete um vetor u num hiperplano no $\mathbb{R}^n$. A transformação tem a forma

$$H_u = I - 2\frac{uu^T}{u^T u}, \tag{10}$$

onde u é um vetor não nulo. Evidentemente, H_u é uma matriz quadrada de ordem n, pois uu^T é uma matriz $n \times n$ e $u^T u$ é um escalar.

Aplicar uma transformação de Householder a um vetor $u \in \mathbb{R}^n$ resulta em $-u$. Consequentemente, $H_u(cu) = -cu$ para todo escalar c. De fato,

$$H_u(u) = \left(I - 2\frac{uu^T}{u^T u}\right) u = u - 2\frac{uu^T u}{u^T u} = u - 2u = -u.$$

Sendo w um vetor no $\mathbb{R}^n$, seja $v = w - \text{proj}_u(w)$. Sabemos que v é ortogonal a u, o que implica que $w = c_1 u + c_2 v$, donde $c_1 = \langle w, u\rangle / \langle u, u\rangle$ e $c_2 = \langle w, v\rangle / \langle v, v\rangle$. Afirmamos que $H_u(w)$ é uma reflexão de w em relação ao hiperplano ortogonal a u. Com efeito,

$$\begin{aligned} H_u(c_1 u + c_2 v) &= \left(I - 2\frac{uu^T}{u^T u}\right)(c_1 u + c_2 v) \\ &= c_1 u + c_2 v - 2\frac{uu^T}{u^T u}(c_1 u + c_2 v) \\ &= -c_1 u + c_2 v - 2c_2 \frac{uu^T}{u^T u} v \\ &= -c_1 u + c_2 v - 2c_2 \frac{u\langle u, v\rangle}{\langle u, u\rangle} \\ &= -c_1 u + c_2 v \end{aligned}$$

Proposição 2. *Seja H_u uma reflexão de Householder com $u \in \mathbb{R}^n$. Então*

1. *H_u é simétrica.*
2. *H_u é ortogonal.*
3. *$H_u^2 = I$.*
4. *$H_u v = v$ se $\langle v, u\rangle = 0$.*

Demonstração. 1. $H_u^T = \left(I - 2\frac{uu^T}{u^T u}\right)^T = I - 2\frac{uu^T}{u^T u} = H_u$.

2. Seja $\beta \overset{\text{def}}{=} 2/(u^T u)$. Então $H_u^T H_u = (I - \beta uu^T)(I - \beta uu^T) = I - 2\beta uu^T + \beta^2 u(u^T u)u^T = I - 2\left(\frac{2}{u^T u}\right)uu^T + \left(\frac{4}{(u^T u)^2}\right)u(u^T u)u^T = I$.

3. $H_u^2 = H_u H_u = I$.

4. Se $\langle v, u\rangle = 0$, então $H_u v = \left(I - 2\dfrac{uu^T}{u^T u}\right) v = v - 2\dfrac{u\langle u, v\rangle}{\langle u, u\rangle} = v$.

□

Uma única reflexão de Householder pode zerar todos os elementos $a_{i+1,i}, a_{i+2,i}, \ldots, a_{mi}$ abaixo da entrada $a_i i$ de uma matriz A. Aplicando uma sequência de reflexões de Householder, podemos transformar A numa matriz triangular-superior, ou seja, queremos transformar

$$A = \begin{bmatrix} X & X & X & X & X & X & \ldots & X \\ X & X & X & X & X & X & \ldots & X \\ \vdots & \vdots & \vdots & \vdots & \vdots & \vdots & \ldots & \vdots \\ X & X & X & X & X & X & \ldots & X \\ X & X & X & X & X & X & \ldots & X \end{bmatrix}$$

na forma

$$H_u A = \begin{bmatrix} X & X & X & X & X & X & \ldots & X \\ 0 & X & X & X & X & X & \ldots & X \\ 0 & 0 & X & X & X & X & \ldots & X \\ 0 & 0 & 0 & X & X & X & \ldots & X \\ 0 & 0 & 0 & 0 & X & X & \ldots & X \end{bmatrix}.$$

Começamos zerando as entradas abaixo de a_{11}. Seja x a primeira coluna de A. Então $H_u x$ será a primeira coluna de $H_u A$. Se $x \neq ke_1 = [k0 \ldots 0]^T$, buscamos um vetor u tal que $H_u x$ tenha zeros por toda parte exceto no índice $(1, 1)$. Os elementos nas colunas restantes serão alterados, mas não nos preocupamos com isso.

A escolha do vetor u é melhor entendida geometricamente. Sabemos que $H_u x$ reflete x através do plano perpendicular a u, e que $\|H_u x\|_2 = \|x\|_2$ porque H_u é ortogonal. Então buscamos u tal que $H_u x = \pm\|x\|_2 e_1$. Em outras palavras, queremos refletir x através dum hiperplano que bisecta o ângulo entre x e e_1. Isso pode ser feito tomando $u = x \pm \|x\|_2 e_1$[6]. O sinal em $u = \pm\|x\|_2 e_1$ é escolhido para evitar cancelamento catastrófico. Como

$$[x_1 x_2 \ldots x_{m-1} x_m]^T \pm \|x\|_2 e_1 = [x_1 \pm \|x\|_2 x_2 \ldots x_{m-1} x_m]^T,$$

apenas a primeira componente de u é afetada pelo sinal escolhido. Escolhemos o sinal de modo que uma adição seja feita.

$$u = \begin{cases} x + \|x\|_2 e_1, & x_1 > 0 \\ x - \|x\|_2 e_1, & x_1 < 0 \\ x + \|x\|_2 e_1, & x_1 = 0 \end{cases}$$

Outro possível problema é o overflow ou underflow no cálculo de $\|x\|_2$. Para minimizar esse risco, podemos normalizar x antes de calcular u e depois multiplicar u por $\|x\|_2$.

1. $\text{colmax} = \max(|x_1|, |x_2|, \ldots, |x_m|)$
2. $\overline{x} = x/\text{colmax}$
3. $\|x\|_2 = \text{colmax}\|\overline{x}\|_2$

Lema. *Se H_u é uma reflexão de Householder e k é uma constante, então $H_{ku} = H_u$.*

[6] O resultado pode ser verificado algebricamente, basta verificar a igualdade $H_u x = \mp\|x\|_2 e_1$

Demonstração. $H_{ku} = I - 2\dfrac{(ku)(ku)^T}{(ku)^T(ku)} = I - 2\dfrac{k^2uu^T}{k^2u^Tu} = I - 2\dfrac{uu^T}{u^Tu} = H_u.$ □

Assim como no caso das rotações de Givens, a multiplicação por uma reflexão de Householder H_u não exige construir a matriz. Se H_u é uma matriz de Householder $m \times m$, x é um vetor no $\mathbb{R}^m$, e A é uma matriz $m \times n$, então os produtos $H_u x$ e $H_u A$ podem ser computador por uma simples fórmula. Se H_u é $n \times n$, o mesmo se aplica para AH_u (usado para zerar linhas de A). Sendo $\beta = 2/(u^Tu)$, temos

$$H_u x = \left(I - 2\frac{uu^T}{u^Tu}\right) x = x - \beta u(u^T x) \tag{11}$$

$$H_u A = \left(I - 2\frac{uu^T}{u^Tu}\right) A = A - \beta u(u^T A) \tag{12}$$

$$AH_u = A\left(I - 2\frac{uu^T}{u^Tu}\right) = A - \beta(Au)u^T \tag{13}$$

Usando a fórmula implícita $A - \beta u(u^T A)$, fazemos uma multiplicação vetor-matriz seguida de uma multiplicação escalar-vetor para computar $H_u A$ em apenas $\mathcal{O}(4mn)$ operações. Se fôssemos construir H_u explicitamente, precisaríamos de $\mathcal{O}(m^2)$ operações para construir H_u e $\mathcal{O}(m^2 n)$ para computar $H_u A$, totalizando $\mathcal{O}(m^2 n)$ operações, o que é muito mais custoso computacionalmente.

O seguinte algoritmo zera todos os elementos abaixo da entrada a_{11} usando uma reflexão de Householder.

Listing 6: Zerando entradas na primeira coluna de uma matriz usando uma reflexão de Householder

```
function [A,u] = hzero1(A)
    // zero out all elements below a_11 in matrix A by a Householder
        reflection H_u
    // input: m x n matrix A
    // output: m x n matrix A implicitly premultiplied by H_u and
        Householder vector u that defines the hyperplane about which x is
         reflected

    x = A(:,1);
    colmax = max(abs(x));
    x = x/colmax; // normalize x to minimize cancellation errors
    colnorm = norm(x,2);
    u = x;

    // construct Householder vector u
    if u(1) >= 0 then
        u(1) = u(1) + colnorm;
    else
        u(1) = u(1) - colnorm;
    end

    // implicitly form H_uA
    unorm = norm(u,2);
    if unorm ~= 0 then
        beta = 2/(unorm^2);
    else
        beta = 0;
    end
```

```
        A = A - (beta*u)*(u'*A);
    endfunction
```

Para zerar os elementos abaixo das diagonais restantes com 6, criamos implicitamente uma sequência de matrizes de Householder que atuem em blocos de submatrizes. Suponha que tenhamos zerado todas as entradas abaixo da diagonal do índice $(1,1)$ ao $(i-1,i-1)$, obtendo uma sequência de matrizes $A_1, A_2, \ldots, A_{i-1}$. Precisamos de uma reflexão de Householder que zere os elementos $\overline{a_{i+1,i}}, \ldots, \overline{a_{mi}}$ e altere apenas elementos da submatriz correspondente de tamanho $(m-i+1, m-i+1)$. Então construímos $\tilde{H}_u$ com o vetor $x = [\overline{a_{ii}}\overline{a_{i+1,j}} \ldots \overline{a_{mi}}]^T$. Essa operação é realizada com o comando

```
[A(i:m,i:n), u] = hzero1(A(i:m,i:n)).
```

e repetida $k = \min(m-1, n)$ vezes para obter a matriz triangular-superior R. O processo assim resulta na fatoração QR da matriz A

$$\tilde{H}_k \tilde{H}_{k-1} \ldots \tilde{H}_1 A = R \tag{14}$$

$$A = (\tilde{H}_1 \tilde{H}_2 \ldots \tilde{H}_k)^T R \tag{15}$$

$$= (\tilde{H}_k \tilde{H}_{k-1} \ldots \tilde{H}_1) R = QR, \tag{16}$$

onde $Q = \tilde{H}_1 \tilde{H}_2 \ldots \tilde{H}_k$ é o produto de matrizes ortogonais, logo ortogonal. Inicializa $Q = I$ como matriz identidade $m \times m$ e computa $Q(1:m, 1:m) = IH_{u1}$, alterando as entradas em toda a matriz Q. Depois computa $Q(1:m, 2:m) = Q(1:m, 2:m)H_{u2}$, alterando as entradas em toda a matriz Q exceto as da primeira coluna, e assim por diante, até $Q(1:m, k:m) = Q(1:m, k:m)H_{uk}$. O algoritmo abaixo realiza a decomposição QR de Householder.

Listing 7: Decomposição QR de Householder

```
function [Q,R] = householderqr(A)
    // compute the QR decomposition of matrix A using Householder
        reflections
    // input: m x n matrix A
    // output: m x m matrix Q and m x n upper-triangular matrix R such
        that A=QR
    R = A;
    [m,n] = size(A)
    Q = eye(m,m);
    k = min(m-1,n);

    for i = 1:k
        [R(i:m,i:n),u] = hzero1(R(i:m,i:n));
        Q(1:m,i:m) = Q(1:m,i:m) - (2/norm(u,2)^2)*(Q(1:m,i:m)*u)*u';
    end
endfunction
```

A contagem de *flops* para o algoritmo de Householder é $2n^2(m - n/3)$ para $m \geq n$. O algoritmo de Givens requer $6n^2(m - n/3)$ *flops*, ou seja, cerca de duas vezes mais do que o algoritmo de Householder.

VI Problemas de mínimos quadrados

1 Existência e unicidade de soluções de mínimos quadrados

2 Resolvendo problemas de mínimos quadrados sobredeterminados

3 Condicionamento de problemas de mínimos quadrados

VII Autovalores e Autovetores

1 Teoremas de Gerschgorin

2 Método da potência para computar autovalor dominante

3 Iteração inversa de Hessenberg

VIII Métodos Iterativos

1 Método de Jacobi

2 Método de Gauss-Seidel

3 Iteração SOR

4 Convergência dos métodos iterativos